

JAVA PROGRAMMING

Chapter 4: Operators in Java

By Akanshu Jamwal

Table of Contents

1. What Are Operators?
2. Types of Operators in Java
3. Arithmetic Operators
4. Assignment Operators
5. Relational Operators
6. Logical Operators
7. Unary Operators
8. Bitwise Operators
9. instanceof Operator
10. Ternary Operator
11. Operator Precedence
12. Why Operators Matter

1. What Are Operators?

Operators are special symbols used to perform operations on variables and values. They are fundamental building blocks of any programming language.

Example:

```
int solvedWeek1 = 40;
int solvedWeek2 = 35;

int totalSolved = solvedWeek1 + solvedWeek2;
```

Here, + is an operator that performs addition on two operands (solvedWeek1 and solvedWeek2).

2. Types of Operators in Java

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Logical Operators
- Unary Operators
- Bitwise Operators
- Other Operators (instanceof, ternary)

■ Why Operators Matter:

Understanding operators deeply is critical for DSA, Competitive Programming, Backend Development, and Coding Interviews.

3. Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric values.

Operator	Name	Example	Result
----------	------	---------	--------

+	Addition	25 + 15	40
-	Subtraction	25 - 15	10
*	Multiply	25 × 4	100
/	Division	25 / 7	3
%	Modulo	25 % 7	4

Key Points:

- Integer Division: $25 / 7 = 3$ (decimal part discarded)
- Modulo is used in: even/odd checks, circular indexing, DSA problems

4. Assignment Operators

Assignment operators assign values to variables. Compound assignments are frequently used in loops.

Op	Equivalent	Example
=	Assign	$x = 10$
+=	$x = x + y$	$x += 5$
-=	$x = x - y$	$x -= 3$
*=	$x = x * y$	$x *= 2$
/=	$x = x / y$	$x /= 4$
%=	$x = x \% y$	$x \% = 5$

5. Relational Operators

Relational operators compare two values and return a boolean result (true or false).

Op	Name	Example (45, 50)	Result
==	Equal	$45 == 50$	false
!=	Not Equal	$45 != 50$	true
>	Greater	$45 > 50$	false

<	Less	45 < 50	true
>=	Greater or Equal	45 >= 50	false
<=	Less or Equal	45 <= 50	true

6. Logical Operators

Logical operators combine boolean expressions and return boolean results.

Op	Name	Example	Result
&&	AND	true && false	false
	OR	true false	true
!	NOT	!false	true

Explanation:

AND (&&): Returns true ONLY if BOTH conditions are true

OR (||): Returns true if AT LEAST ONE condition is true

NOT (!): Reverses the boolean value

7. Unary Operators

Unary operators operate on a single operand. Increment/Decrement operators are frequently asked in interviews.

Op	Name	Example	Explanation
++	Pre-increment	++x	Increment first, then use
++	Post-increment	x++	Use first, then increment
--	Pre-decrement	--x	Decrement first, then use
--	Post-decrement	x--	Use first, then decrement

Example to understand the difference:

```
int x = 100;
int prefix = ++x;    // x becomes 101, then assign
int postfix = x++;   // assign first (101), then increment

System.out.println(prefix); // 101
System.out.println(postfix); // 101
System.out.println(x);      // 102
```

■■ Interview Alert: This difference is frequently asked in coding interviews!

8. Bitwise Operators

Bitwise operators operate at binary level and are heavily used in advanced DSA and competitive programming.

Op	Name	Example	Result
&	AND	6 & 3	2
	OR	6 3	7
^	XOR	6 ^ 3	5
~	NOT	~6	-7
<<	Left Shift	6 << 1	12
>>	Right Shift	6 >> 1	3

Used in: Advanced DSA, Competitive Programming, Optimization Problems

9. instanceof Operator

The instanceof operator checks whether an object belongs to a specific class or interface.

```
String track = "CodeHelp ONE";

boolean result = track instanceof String;
System.out.println(result); // true
```

10. Ternary Operator

The ternary operator is a shorthand for if-else statements.

Syntax:

```
condition ? valueIfTrue : valueIfFalse
```

Example:

```
int solved = 320;

String level = (solved >= 300) ? "Advanced" : "Intermediate";
System.out.println(level); // Output: Advanced
```

11. Operator Precedence

Operators are evaluated in a specific order. Understanding precedence prevents logical bugs.

Example:

```
int result = 10 + 5 * 2;
// Multiplication happens BEFORE addition
// = 10 + 10 = 20 (NOT 30)
```

Precedence Order (High to Low):

1. () [] . (Parentheses, brackets)
2. ++ -- ! ~ (Unary operators)
3. * / % (Multiplication, Division)

4. + - (Addition, Subtraction)
5. << >> (Shift operators)
6. < <= > >= (Relational)
7. == != (Equality)
8. & (Bitwise AND)
9. ^ (Bitwise XOR)
10. | (Bitwise OR)
11. && (Logical AND)
12. || (Logical OR)
13. ?: = += -= (Assignment, Ternary)

■ **Tip:** When in doubt about precedence, use parentheses to make your intent clear!

12. Why Operators Matter

Most coding round mistakes happen due to operator-related errors:

- Integer Division Confusion: $25 / 7 = 3$ (not 3.57)
- Wrong Logical Conditions: Using `||` instead of `&&`
- Prefix/Postfix Misunderstanding: `++x` vs `x++`
- Incorrect Precedence: Wrong calculation order
- Type Mismatch Issues: Using incompatible types

■ Key Takeaway:

Strong operator clarity improves accuracy significantly. Practice using different operators in various scenarios. Pay special attention to edge cases and operator precedence!